

Sprint 3

Nivel 1: Entorno e Ingesta Híbrida (Code-First)

Escenario:

Acaba de llegar a la empresa. El equipo de Ingeniería de Datos ha dejado los archivos históricos en el Data Lake corporativo.

Tu primera misión es diseñar y crear el entorno de trabajo. El CTO exige profesionalidad: no volcaremos datos sin orden, sino que implementaremos una Arquitectura Medallion.

Ejercicio 1: Arquitectura de Datos (Lógica vs. Física)

Antes de tocar la consola, debes entender la diferencia entre lo que diseñamos y lo que construimos:

- » **Arquitectura Lógica (Capas de Datos)** : Es el diseño conceptual. Organiza los datos según su nivel de calidad (Bronze, Silver, Gold).
- » **Implementación Física (Datasets)** : Es el recurso real a Google Cloud. En BigQuery, utilizaremos Datasets (contenedores) para materializar físicamente estas capas lógicas.

🧩 Tareas:

Implementará físicamente la arquitectura lógica creando tres Datasets dentro de un mismo proyecto para centralizar la facturación y los permisos.

👉 Crea un **Nuevo Proyecto** en Google Cloud llamado:

`sprint3-analytics-[el-teu-nom]`.

Puedes crearlo por la UI, pero si lo haces por Consola (gcloud CLI) tiene un plus.

Para demostrar tu dominio total de la plataforma, crearás cada uno de los datasets utilizando un método distinto:

- » **Dataset Físico** `sprint3_bronze` → Capa Lógica Bronce (Datos Crudas)
 - Función: Datos crudos tal y como llegan de la fuente.
 - Método: Utiliza la **Interfaz Gráfica (UI)** de BigQuery (haciendo clics).
- » **Dataset Físico** `sprint3_silver` → Capa Lógica Silver (Datos Limpios)

- Función: Datos limpios, tipados y deduplicados.
- Método: Utiliza código SQL (CREATE SCHEMA).

» **Dataset Físico** sprint3_gold → Capa Lógica Gold (Datos de Negocio)

- Función: Datos agregados listos para informes y paneles de control (dashboards).
- Método: Utiliza Cloud Shell (Línea de comandos bq).

Tenga en cuenta la ubicación de los datos originales para decidir la ubicación física de los tres Datasets.

Nota : Aunque sean capas lógicas diferentes, físicamente deben vivir en la misma región para poder "hablar" entre ellas sin costes de transferencia.

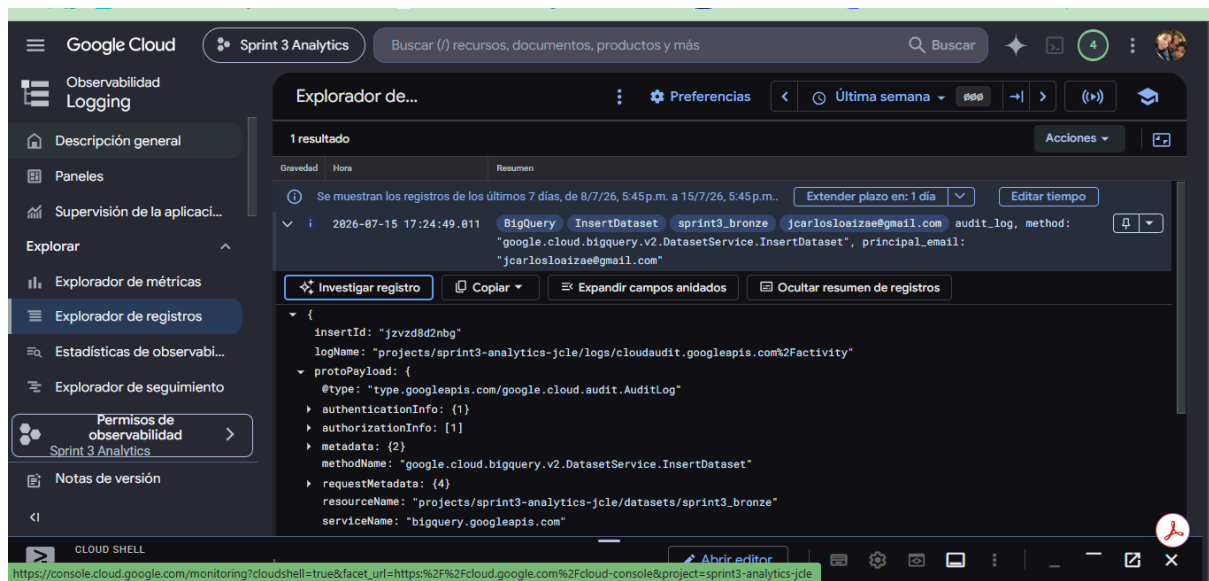
Creación del proyecto mediante la consola (gcloud CLI):

```

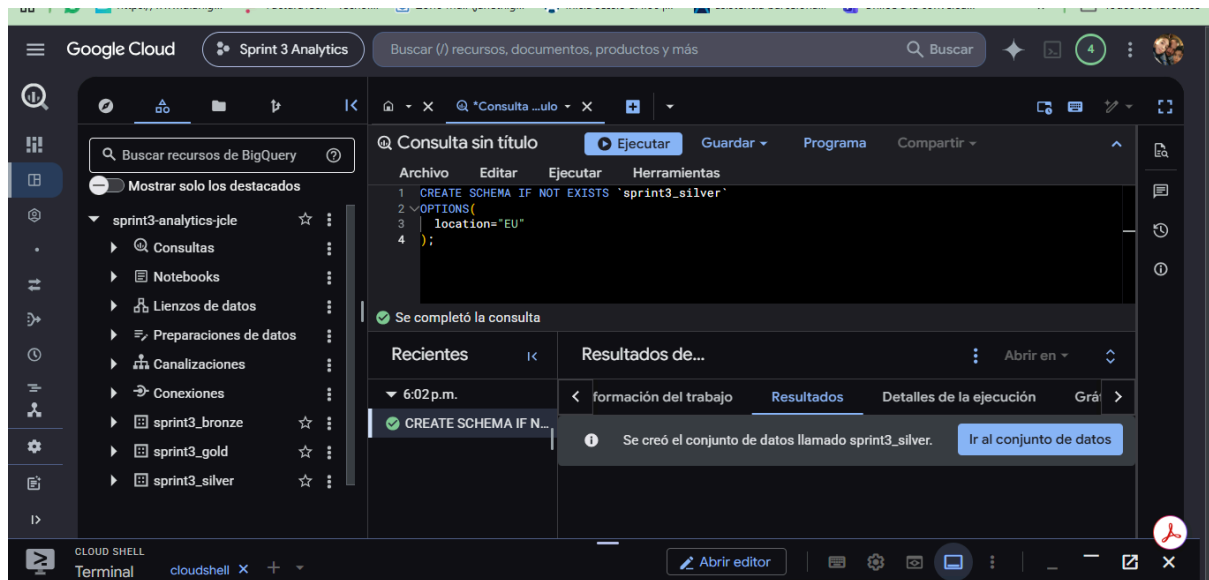
also
Waiting for [operations/create_project.global.908355805622581850] to finish...done.
Enabling service [cloudapis.googleapis.com] on project [sprint3-analytics-jc1e]...
Regional Access Boundary HTTP request failed after retries: response_data={'error': {'code': 404, 'message': 'Not found; Gaia id not found for email jcar
loaizae@gmail.com', 'status': 'NOT_FOUND'}}, retryable_error=False
Regional Access Boundary HTTP request failed after retries: response_data={'error': {'code': 404, 'message': 'Not found; Gaia id not found for email jcar
loaizae@gmail.com', 'status': 'NOT_FOUND'}}, retryable_error=False
Regional Access Boundary HTTP request failed after retries: response_data={'error': {'code': 404, 'message': 'Not found; Gaia id not found for email jcar
loaizae@gmail.com', 'status': 'NOT_FOUND'}}, retryable_error=False
Operation "operations/acet.p2-55048958621-a593620e-92d5-4c3a-8a55-61dc0c3ab725" finished successfully.
jcarlosloaizae@cloudshell:~$ gcloud config set project sprint3-analytics-jc1e
Regional Access Boundary HTTP request failed after retries: response_data={'error': {'code': 404, 'message': 'Not found; Gaia id not found for email jcar
loaizae@gmail.com', 'status': 'NOT_FOUND'}}, retryable_error=False
Updated property [core/project].
jcarlosloaizae@cloudshell:~$ (sprint3-analytics-jc1e)$
jcarlosloaizae@cloudshell:~$ (sprint3-analytics-jc1e)$ gcloud services enable bigquery.googleapis.com
Regional Access Boundary HTTP request failed after retries: response_data={'error': {'code': 404, 'message': 'Not found; Gaia id not found for email jcar
loaizae@gmail.com', 'status': 'NOT_FOUND'}}, retryable_error=False
jcarlosloaizae@cloudshell:~$ (sprint3-analytics-jc1e)$ ^C
jcarlosloaizae@cloudshell:~$ (sprint3-analytics-jc1e)$ bq --location=EU mk --dataset sprint3_gold
Dataset 'sprint3-analytics-jc1e:sprint3_gold' successfully created.
jcarlosloaizae@cloudshell:~$ (sprint3-analytics-jc1e)$

```

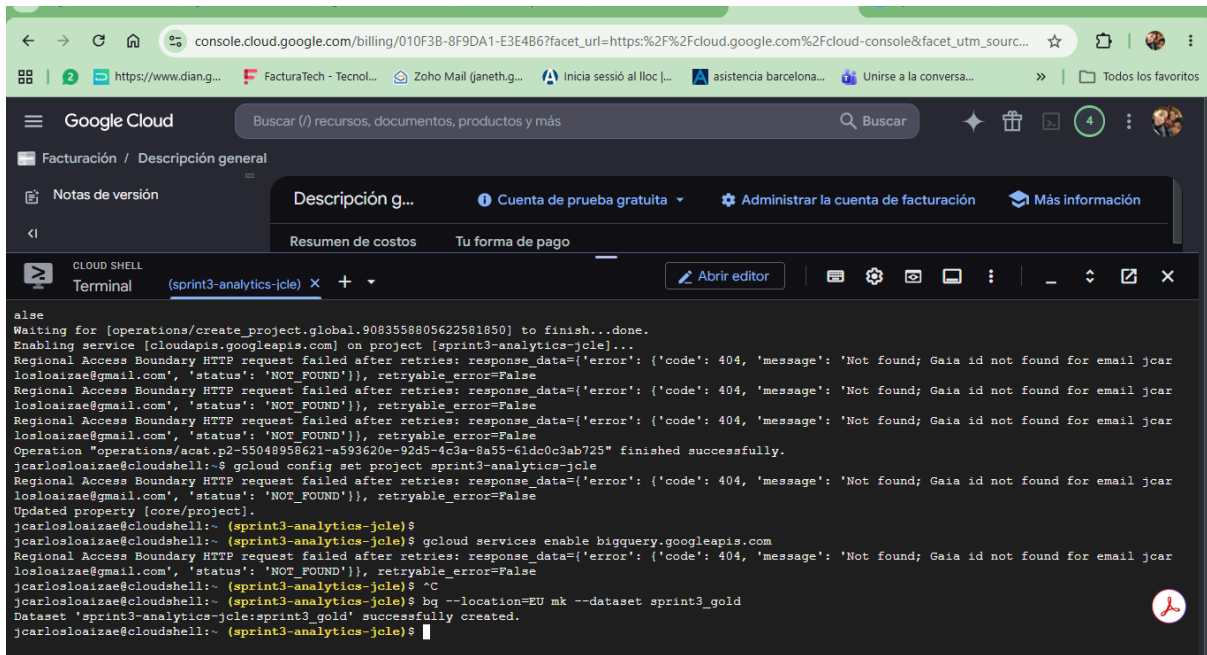
Creacion capa de Bronze mediante la interfaz UI:



Creacion capa Silver con codigo SQL:



Creación de la capa Gold mediante la consola (gcloud CLI):



Ejercicio 2: Ingesta en Capa Bronze (Conexión DDL)

Un analista profesional automatiza las labores. Ahora que tienes tu arquitectura a punto, poblaremos la primera capa (**Bronze**) sin mover los datos originales, creando **Tablas Externas** que apunten al Data Lake.

Tarea:

Escribe y ejecuta las sentencias `CREATE EXTERNAL TABLE` para conectar los siguientes archivos al *dataset* **sprint3_bronze** . Presta mucha atención a las "Notas Técnicas", ya que no todos los archivos tienen el mismo formato.

Mesa	Dominio	Ruta en la Nube (URI)	Notas Técnicas
transactions_raw	ERP	gs://bootcamp-data-analytics-public/ERP/transactions.csv	Delimitador: ; (Punto y coma)
companies_raw	ERP	gs://bootcamp-data-analytics-public/ERP/companies.csv	Cabecera: Ignora la 1a fila. Define manualmente el esquema porque todo es texto.

american_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/american_users.csv	Estándar.
european_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/european_users.csv	Estándar.
credit_cards_raw	CRM	gs://bootcamp-data-analytics-public/CRM/credit_cards.csv	Estándar.

👉 Entrega:

El código SQL utilizado para `transactions_raw` y una captura de pantalla con las 5 tablas creadas.

Código SQL utilizado para `transactions_raw`:

The screenshot shows the Google Cloud BigQuery console interface. The main editor displays the following SQL query:

```
1 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.transactions_raw`
2 OPTIONS (
3   format = 'CSV',
4   uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],
5   field_delimiter = ',';
6 );
```

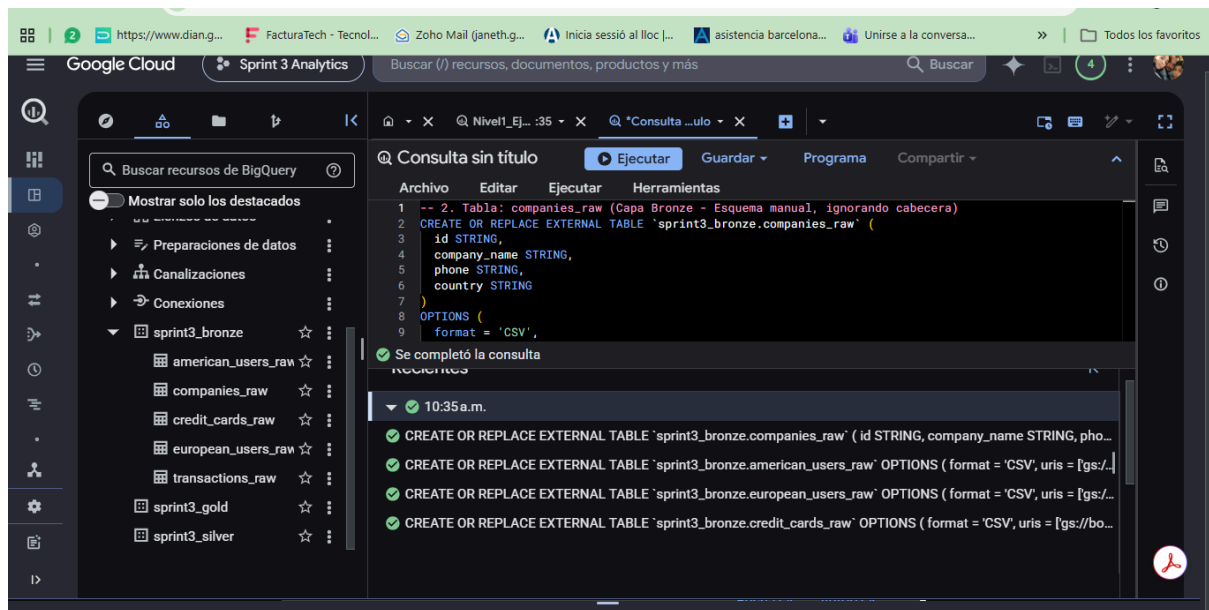
Below the query editor, a message states: "Esta consulta procesará 0 B cuando se ejecute." (This query will process 0 B when executed). The "Recientes" (Recent) section shows the query "CREATE OR REPLACE EXT...". The "Resultados de..." (Results of...) section displays the following job information:

Información del trabajo	Resultados	Detalles de la ejecución
Usuario	jcariosloaizae@gmail.com	
Ubicación	EU	
Hora de creación	15 jul 2026, 6:24:11 p.m. UTC+2	
Hora de inicio	15 jul 2026, 6:24:12 p.m. UTC+2	
Hora de finalización	15 jul 2026, 6:24:13 p.m. UTC+2	
Duración	1 s 8 ms	

link:

<https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1ssprint3-analytics-jcle!2seurope-southwest1!3sa55ac819-1b5f-4cde-a5a2-ba0b652be04a!2e1>

Captura con las tablas creadas:

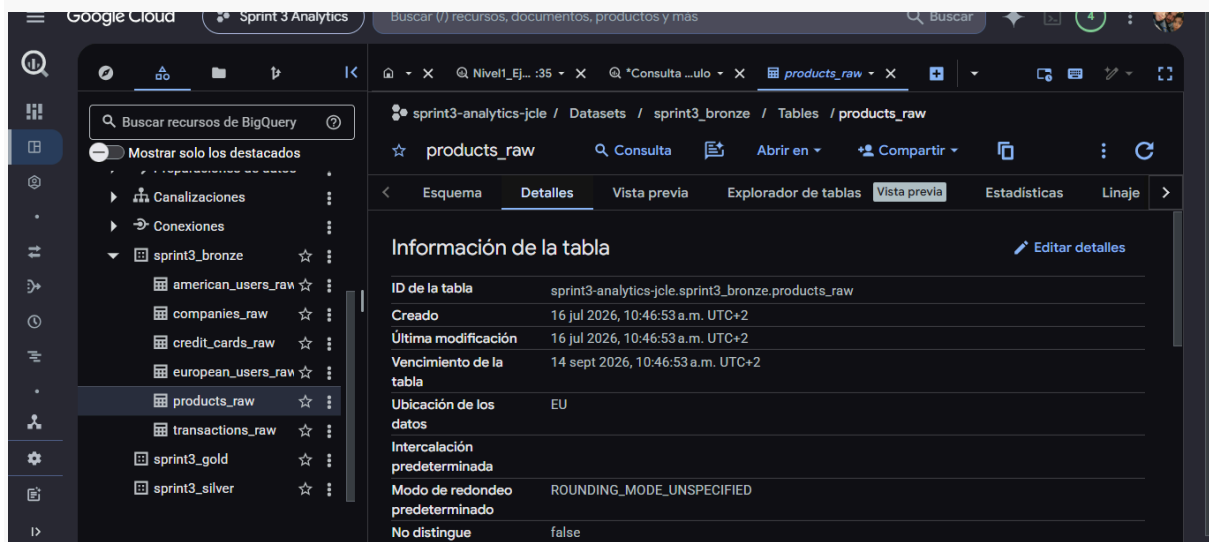


Ejercicio 3: Carga de Datos Locales (Upload)

Falta el catálogo de **Productos** . Esta información no está en Data Lake, utiliza el archivo *products.csv* del sprint 2.

- » Carga este archivo manualmente ("Upload") en el dataset `sprint3_bronze` crea la tabla `products_raw`.
- » Observa que ésta será la única **Mesa Nativa** de momento.

Creación Tabla Nativa:



Ejercicio 4: Arquitectura y Rendimiento. Materialización de Datos (Asistido por IA)

Escenario:

Las consultas sobre el Data Lake van lentas. Tienes que demostrar a tu manager la diferencia de rendimiento entre trabajar con archivos externos y trabajar con tablas nativas de BigQuery.

a) Materialización de Datos (Asistido por IA)

🧩 Tarea:

Crea una tabla nueva llamada `sprint3_bronze.transactions_raw_native` que sea una copia exacta de tu tabla externa `transactions_raw`.

Requisito de "AI-Pair Programming":

No escribas el código SQL manualmente. Utiliza el Asistente de IA (Gemini) integrado en BigQuery (o el chat lateral) para generar la sentencia.

👉 **Tu Rol** : Eres el responsable de validar que el código que te da la IA es correcto antes de ejecutarlo.

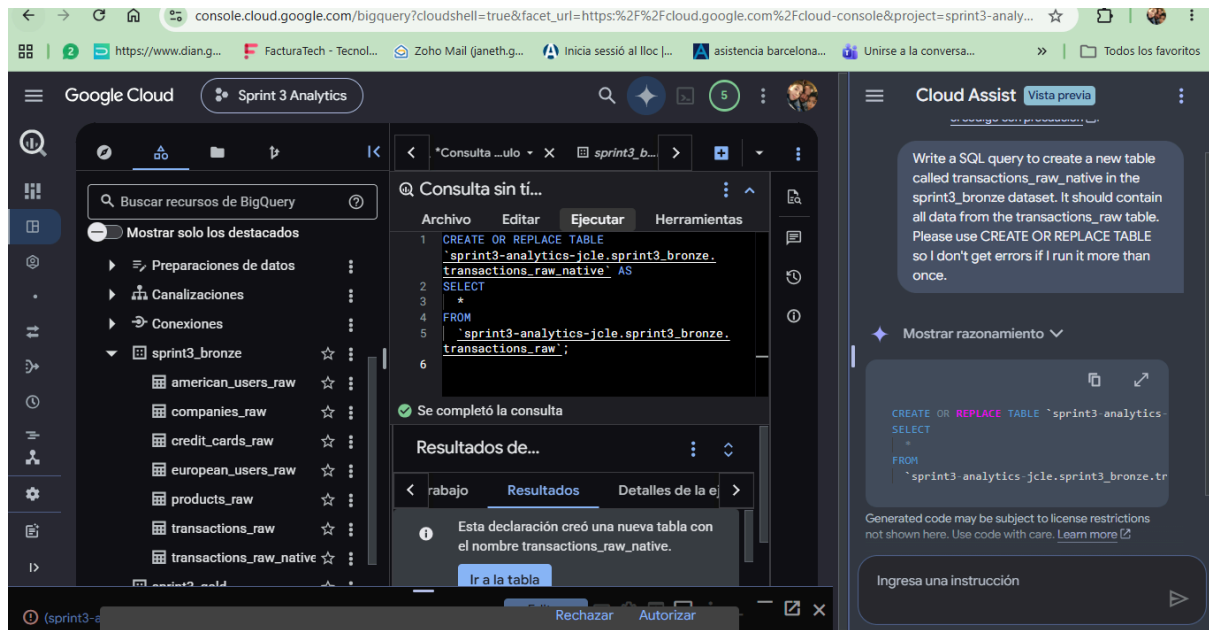
Prompt Sugerido:

"Write a SQL query para crear en el nuevo panel callejera `transactions_raw_native` en el `sprint3_bronze` dataset. Es necesario contain all data from the `transactions_raw` table. Please use sonido `CREATE OR REPLACE TABLE` y no se pueden errores y se run it more than once."

👉 Entrega:

Una captura de pantalla donde se vea:

- » El prompt que escribiste en la IA.
- » El código SQL que la IA va a generar.
- » El resultado de la ejecución ("This statement created a new table...").



b) Auditoría de Costes

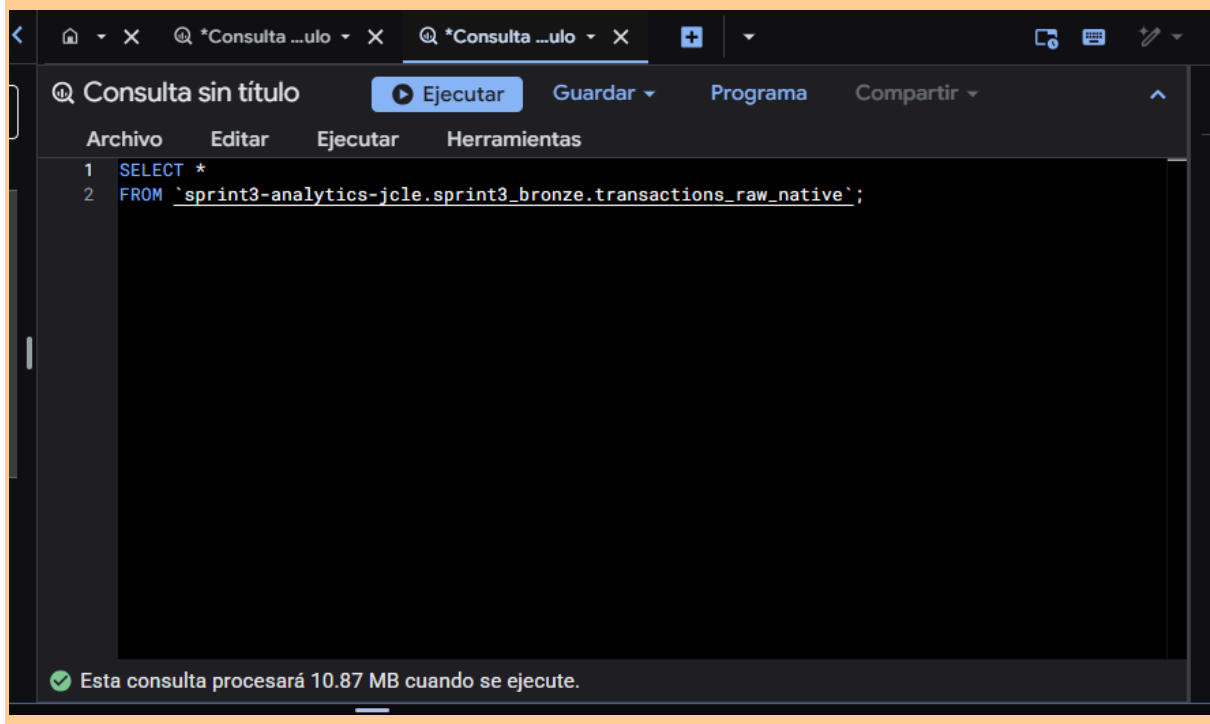
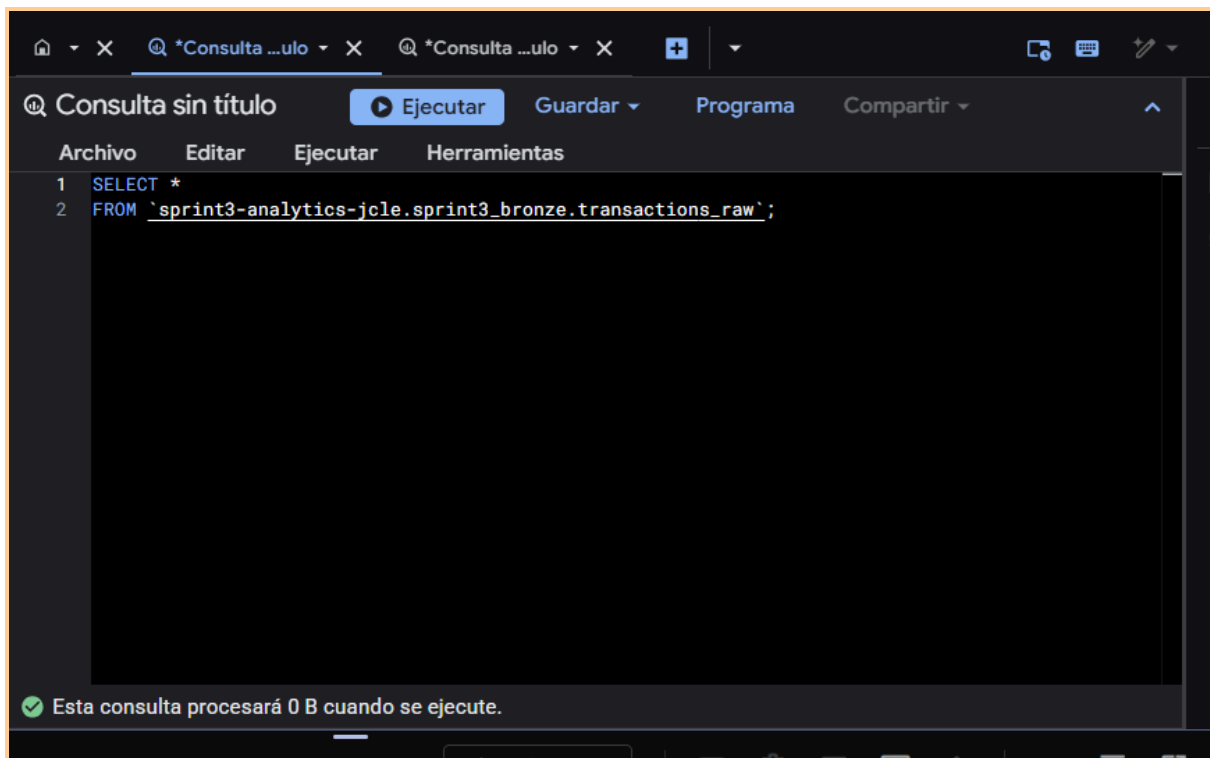
🧩 Tarea:

Diseñar una prueba para comparar el coste de leer una sola columna (ej: id) en la tabla externa vs. la mesa nativa. Utiliza los Bytes procesados y los Bytes facturados para realizar la comparación.

- » ¿Cuánto "pesa" la consulta en `transactions_raw`?
- » ¿Cuánto "pesa" la misma consulta en `transactions_raw_native`?

👉 Entrega:

Capturas del validador mostrando la diferencia y una conclusión técnica.



Mi análisis técnico (Tabla Externa vs. Tabla Nativa)

- **Lo que pasa con la Tabla Externa (0 B):** Al principio me sorprendió ver que marcaba 0 Bytes, pero investigando entendí el truco. Como es un archivo CSV suelto guardado en el Data Lake (Cloud Storage),

BigQuery no tiene ni idea de cuánto pesa ni qué hay dentro hasta que le das al botón de ejecutar. No es que sea gratis, es que va "a ciegas". Al arrancar la consulta, se ve obligado a leer el archivo entero desde fuera fila por fila, lo que acaba siendo más lento y costoso.

- **Lo que pasa con la Tabla Nativa** (10.87 MB): Aquí cambia como importamos los datos directamente "a casa" (al almacenamiento propio de BigQuery), el sistema ya ha escaneado el archivo y sabe exactamente que la tabla pesa 10.87 MB.
- **Mi conclusión como Analista:** Traer los datos a tablas nativas es mucho mejor para trabajar en el día a día. Al estar en su propio formato, si mañana solo necesito consultar una columna (como los IDs), BigQuery solo me va a cobrar por el trocito de esa columna (bajando el peso a menos de 1 MB). En cambio, si la dejara como tabla externa, tendría que tragarse y cobrarme los 10.87 MB enteros cada vez que haga un filtro.

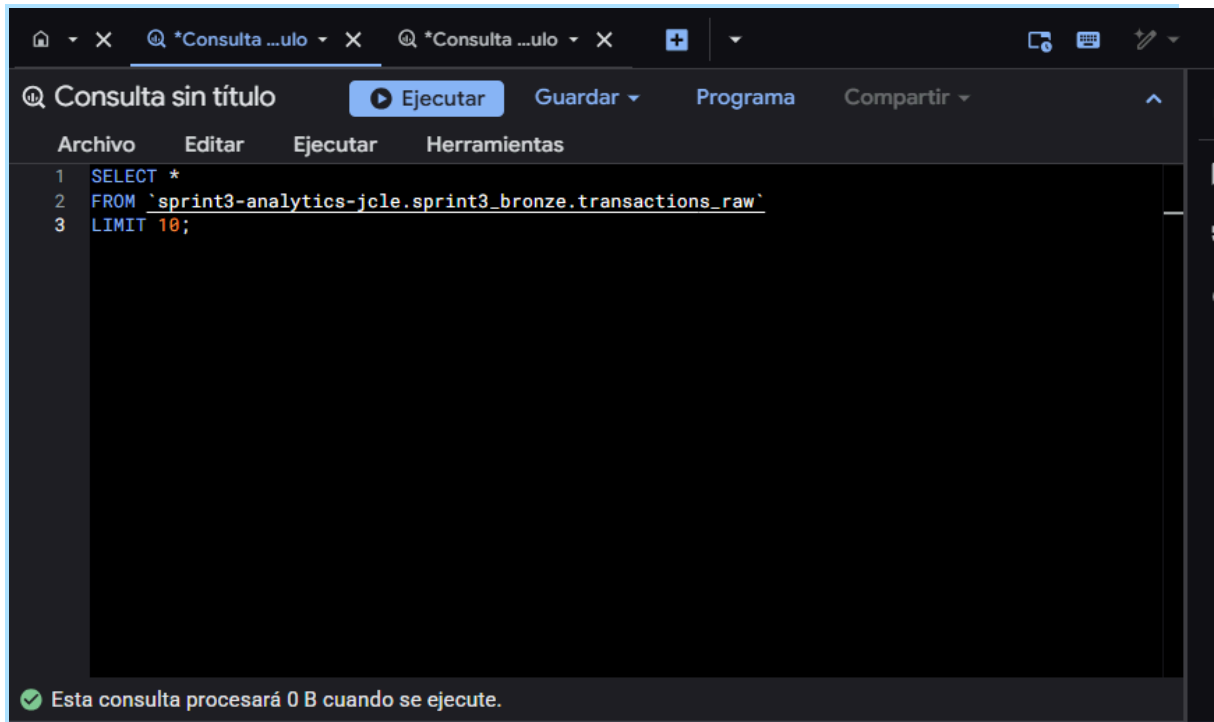
c) El peligro del LIMIT

¿LIMITE ahorra datos procesados en BIGQUERY?

Tarea:

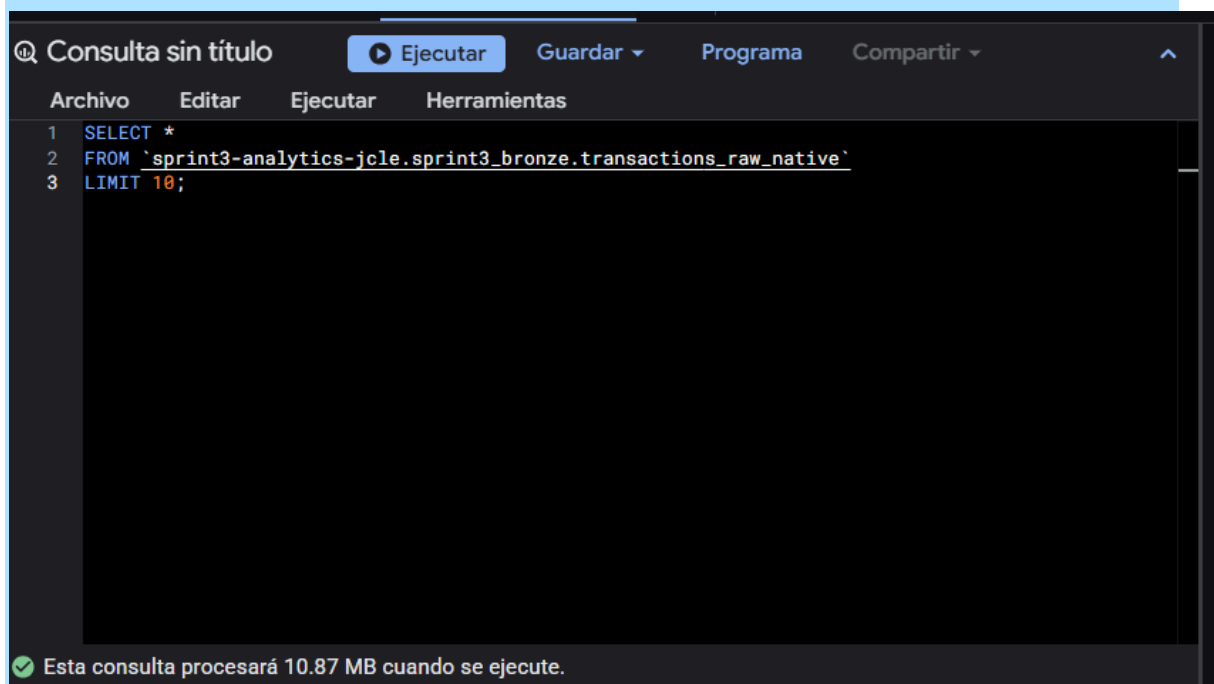
Fes un **LIMIT 10** sobre la taula `transactions_raw_native` y la taula `transactions_raw`

✓ **Comprobación** : Mira el "Dry Run". ¿Ha bajado el coste respecto a no poner LIMIT? Explica brevemente por qué esto es una trampa para principiantes.



```
1 SELECT *
2 FROM `sprint3-analytics-jcle.sprint3_bronze.transactions_raw`
3 LIMIT 10;
```

✓ Esta consulta procesará 0 B cuando se ejecute.



```
1 SELECT *
2 FROM `sprint3-analytics-jcle.sprint3_bronze.transactions_raw_native`
3 LIMIT 10;
```

✓ Esta consulta procesará 10.87 MB cuando se ejecute.

Al principio uno piensa: "Bueno, si solo le pido que me muestre 10 filas con un LIMIT 10, BigQuery trabajará menos y me cobrará casi nada". ¡Pues resulta que es una trampa total para principiantes!

El validador me sigue marcando los mismos 10.87 MB. Esto pasa porque a BigQuery no le importa cuántas filas te pinta en la pantalla al final; a él lo

que le cuesta dinero es ir a buscar los datos al disco. Para poder elegir cuáles son esas 10 filas que te va a mostrar, primero se tiene que procesar y escanear la tabla entera de arriba a abajo.

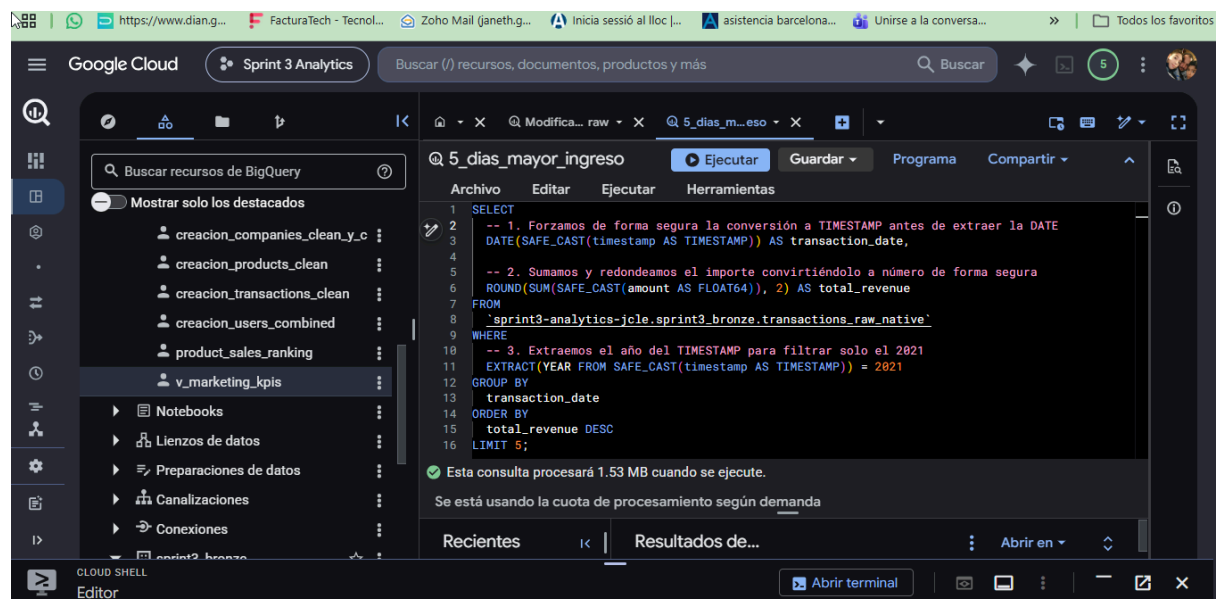
En conclusión: el LIMIT va muy bien para que la pantalla no se colapse con millones de filas, pero no te ahorra ni un céntimo. La única forma real de ahorrar aquí es dejar de usar el SELECT * y pedirle solo las columnas que vayas a usar de verdad.

Ejercicio 5: Adaptación de Sintaxis (Reporting)

Tu jefe quiere saber cuáles fueron los **5 días con mayores ingresos del año 2021**.

🧠 **Reto:**

Probablemente el campo timestamp es un STRING. Tendrás que investigar funciones de BigQuery (SUBSTR, CAST, PARSE_TIMESTAMP) para filtrar el año y agrupar por fecha correctamente



The screenshot shows the Google Cloud BigQuery console. On the left, there's a sidebar with a search bar and a list of datasets including 'v_marketing_kpis'. The main area displays a SQL query in the 'Q 5_dias_mayor_ingreso' editor. The query is as follows:

```
1 SELECT
2 -- 1. Forzamos de forma segura la conversión a TIMESTAMP antes de extraer la DATE
3   DATE(SAFE_CAST(timestamp AS TIMESTAMP)) AS transaction_date,
4
5 -- 2. Sumamos y redondeamos el importe convirtiéndolo a número de forma segura
6   ROUND(SUM(SAFE_CAST(amount AS FLOAT64)), 2) AS total_revenue
7 FROM
8   `sprint3-analytics-jcle.sprint3-bronze.transactions_raw_native`
9 WHERE
10  -- 3. Extraemos el año del TIMESTAMP para filtrar solo el 2021
11   EXTRACT(YEAR FROM SAFE_CAST(timestamp AS TIMESTAMP)) = 2021
12 GROUP BY
13   transaction_date
14 ORDER BY
15   total_revenue DESC
16 LIMIT 5;
```

Below the query, a status message indicates: 'Esta consulta procesará 1.53 MB cuando se ejecute. Se está usando la cuota de procesamiento según demanda'.

Resultado



5_dias_mayor_ingreso Ejecutar Guardar Programa Compartir

Archivo Editar Ejecutar Herramientas

1 SELECT

✓ Esta consulta procesará 1.53 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

> Resultados de... View all jobs Guardar los resultados Abrir en

< Información del trabajo Resultados Visualización JSON Detalles de la ejecución >

Fila	transaction_date	total_revenue	
1	2021-10-27	11440.81	
2	2021-01-22	11328.84	
3	2021-02-11	11182.16	
4	2021-04-29	11119.01	
5	2021-03-27	10898.51	

Resultados por página: 50 1 - 5 de 5 < >

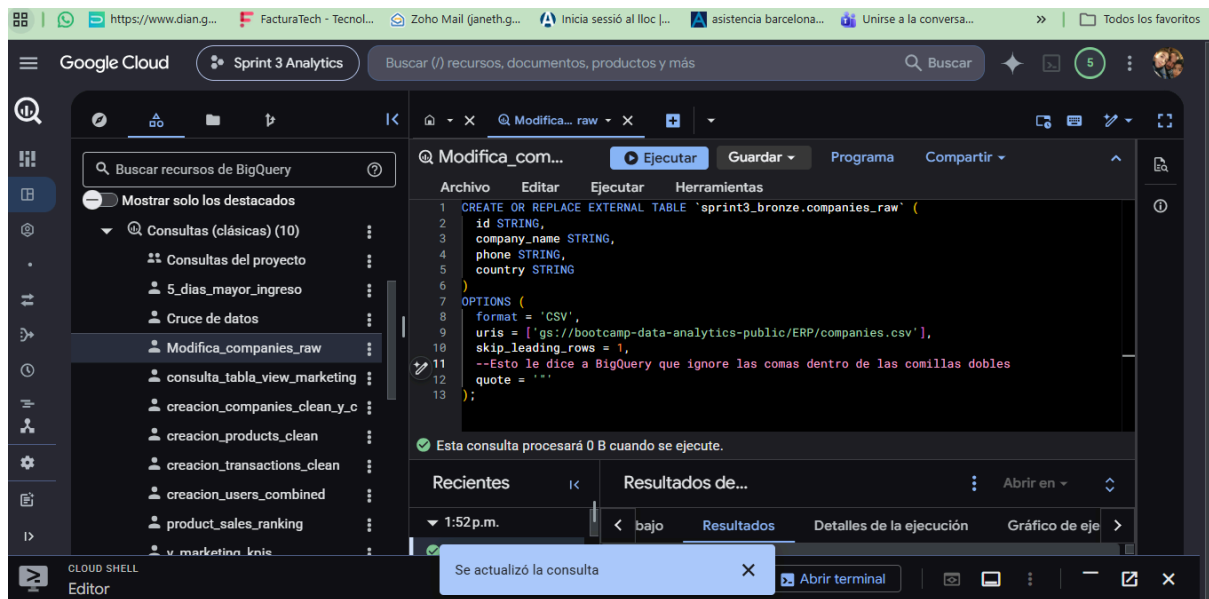
Ejercicio 6: Consultas Complejas

Necesitamos un informe que cruce datos.

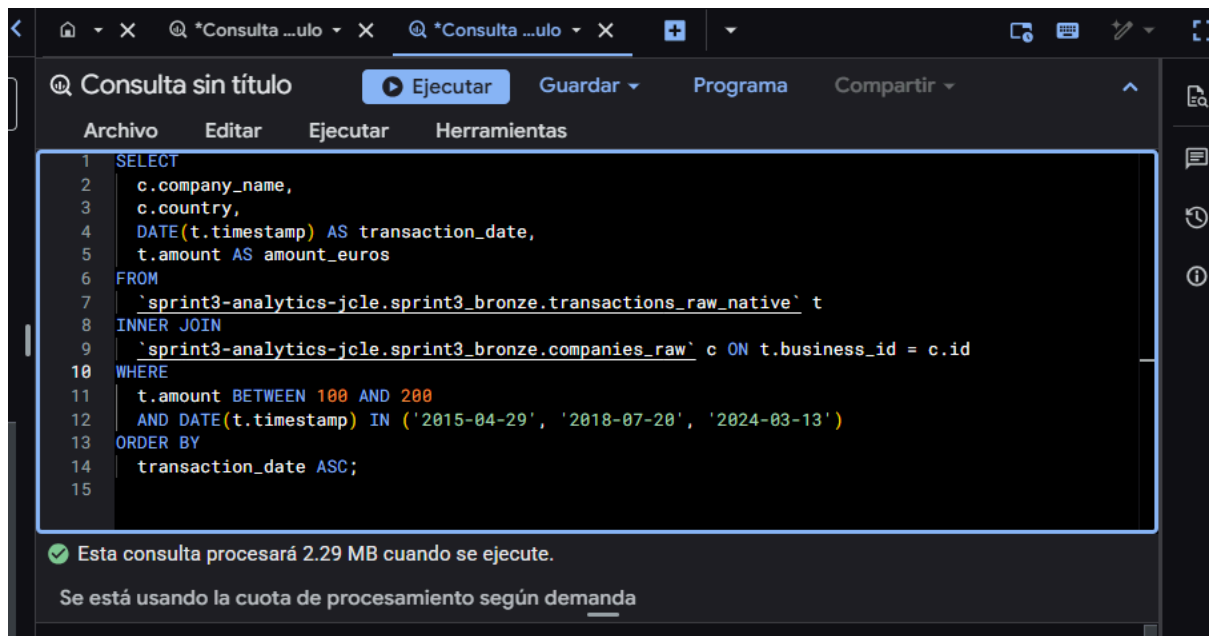
🧩 Tarea:

Lista el nombre, país y fecha de las transacciones realizadas por empresas que realizaron operaciones entre 100 y 200 euros en alguna de estas fechas: 29-04-2015, 20-07-2018 o 13-03-2024.

En esta consulta se encontró un error al cargar la tabla externa `companies_raw` se dijo en BigQuery que tenía 4 columnas y en la línea 2 del archivo físico en Google Cloud Storage, el motor se ha encontrado con 6 columnas. Esto ocurrió porque en el campo que contiene el nombre de las empresas, algunas empresas tienen una coma (,) dentro de su propio nombre (por ejemplo: "Empresa de Transportes, S.A."), Bigquery se confunde y rompe la línea en nuevas columnas, para solucionarlo se debe incluir la línea de código `quote = '"'`, esto le dice a BigQuery que, Si ve una coma, úsala para separar columnas, a menos que esa coma esté dentro de unas comillas dobles.



Cruce de datos:



Resultado del cruce de datos:

Buscar (/) recursos, documentos, productos y más

Resultados de... View all jobs Guardar los resultados Abrir en

Información del trabajo Resultados Visualización JSON Detalles de la ejecución

Fila	company_name	country	transaction_date	amount_euros
1	Dictum Eu Corp.	donec.vitae@icloud.ca	2015-04-29	177.67
2	Auctor Mauris Vel LLP	nec.tempus@icloud.couk	2015-04-29	181.42
3	Donec Fringilla PC	ut.tincidunt@hotmail.ca	2015-04-29	187.52
4	Non Magna LLC	nisl.quisque.fringilla@hotmail.ca	2015-04-29	178.1
5	Ac Libero Inc.	mollis.lectus@protonmail.ca	2015-04-29	124.04
6	Amet Lorem LLP	ultrices.adipiscing@icloud.edu	2015-04-29	145.89
7	Tempor Diam Institute	cursus.in@yahoo.com	2018-07-20	124.04
8	Tempor Diam Institute	cursus.in@yahoo.com	2018-07-20	124.04
9	Et Magnis Ltd	dapibus.ligula@protonmail.net	2018-07-20	114.44
10	Justo Eu Arcu Ltd	ac.facilisis@icloud.couk	2018-07-20	136.6

Resultados por página: 50 1 - 19 de 19

Rechazar Autorizar

Nivel 2: Limpieza y Transformación (ELT)

Escenario:

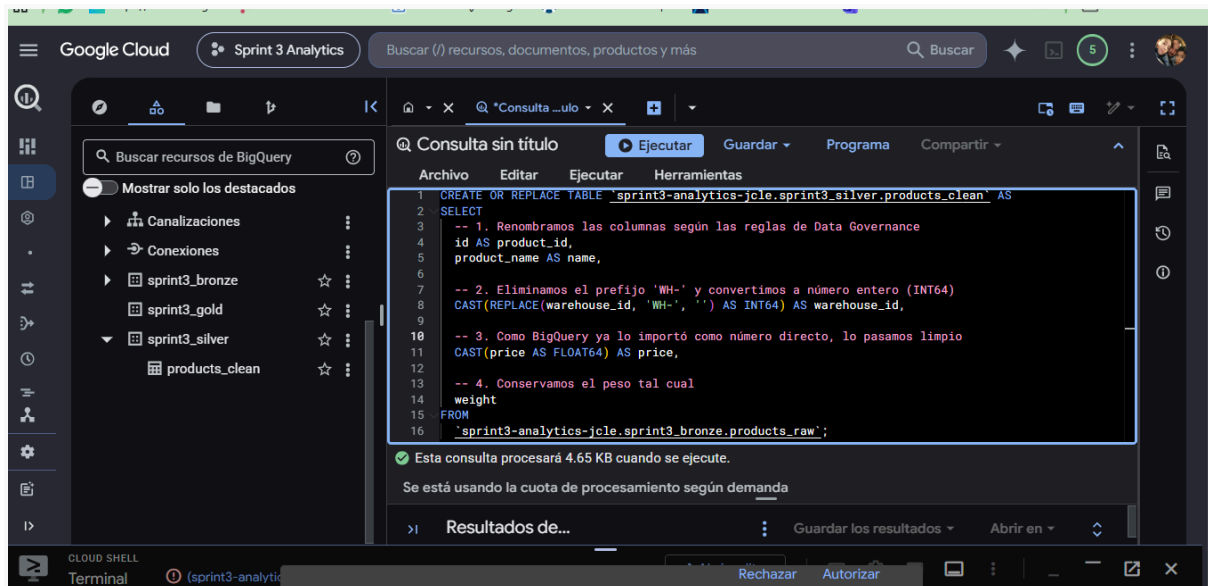
Los datos brutos tienen problemas de calidad (símbolos de moneda, usuarios duplicados, fechas incorrectas). Actúa como Data Engineer para limpiarlo todo.

Ejercicio 1: Limpieza de Productos (Data Quality)

Crearemos la capa limpia ("Silver") para los productos. Tienes la mesa `sprint3_bronze.products_raw` cargada desde el CSV. El equipo de Data Governance te pide crear una mesa de productos limpia a `sprint3_silver.products_clean` que cumpla estas reglas de calidad:

- » **Estandarización de Nombres** : La columna original `id` es ambigua; renombrarla a `product_id`. La columna `product_name` simplifícala a `name`.
- » **Limpieza de IDs** : El campo `warehouse_id` tiene el formato antiguo "WH-4". Elimina el prefijo "WH-" y convierte el valor en número entero (INT64).

- » **Garantía de Precio** : Asegúrate de que price es un número (FLOAT64), sin símbolos de moneda.
- » **Demás columnas** : Conserva el campo weight (peso) tal cual.



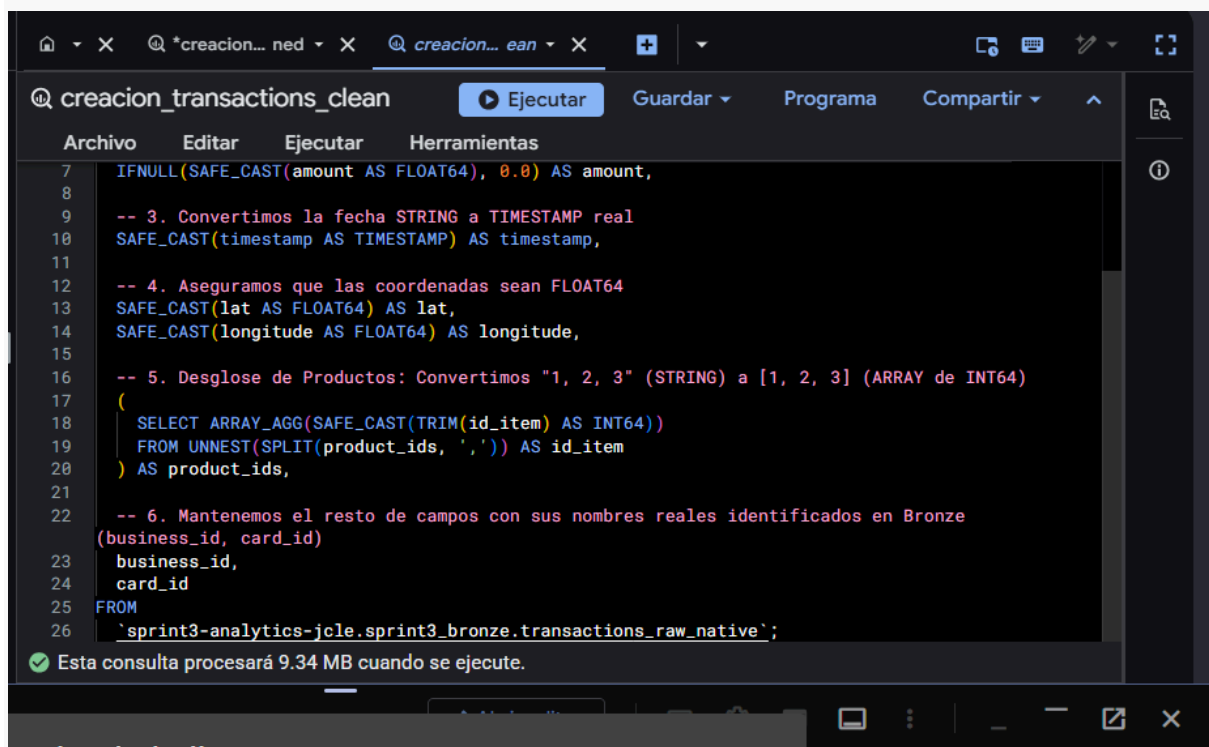
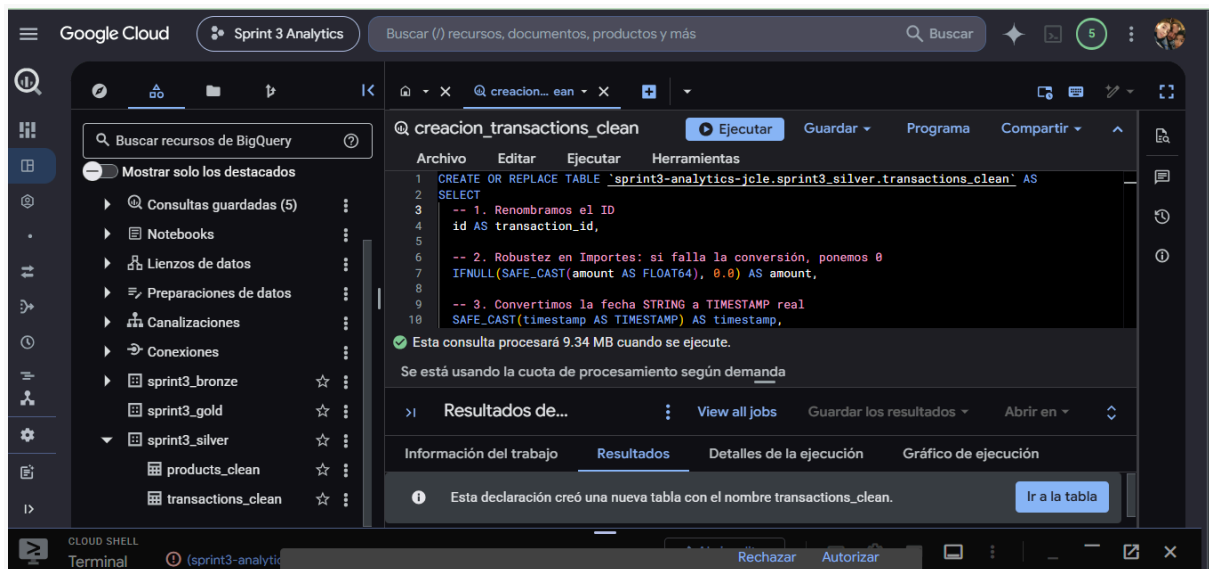
Ejercicio 2: Creación de Transacciones Limpias (Capa Silver)

Escenario:

La tabla transactions_raw (Bronze) es insegura: el importe (amount) podría tener letras, las coordenadas podrían fallar y la fecha es un texto difícil de filtrar. Crearemos la mesa definitiva

sprint3_silver.transactions_clean que garantice lo siguiente.

- » **Estandarización de Nombres** : La columna original id es ambigua; renombrarla a transaction_id.
- » **Robustez en Importes** : Utiliza SAFE_CAST en el campo amount. Si falla, sustitúyelo por 0 (usando IF NULL).
- » **Fechas Reales** : Convierte el campo timestamp (que es STRING) a tipo TIMESTAMP real.
- » **Coordenadas** : Asegura que lat y longitud sean FLOAT64 (utiliza SAFE_CAST para seguridad).
- » **Desglose de Productos**: Transforma la cadena de texto **product_ids** en un **ARRAY de enteros** .
 - Input: "1, 2, 3" (String)
 - Output: [1, 2, 3] (Array/Lista de enteros)
- » **Resto de campos** : Manténlos igual.



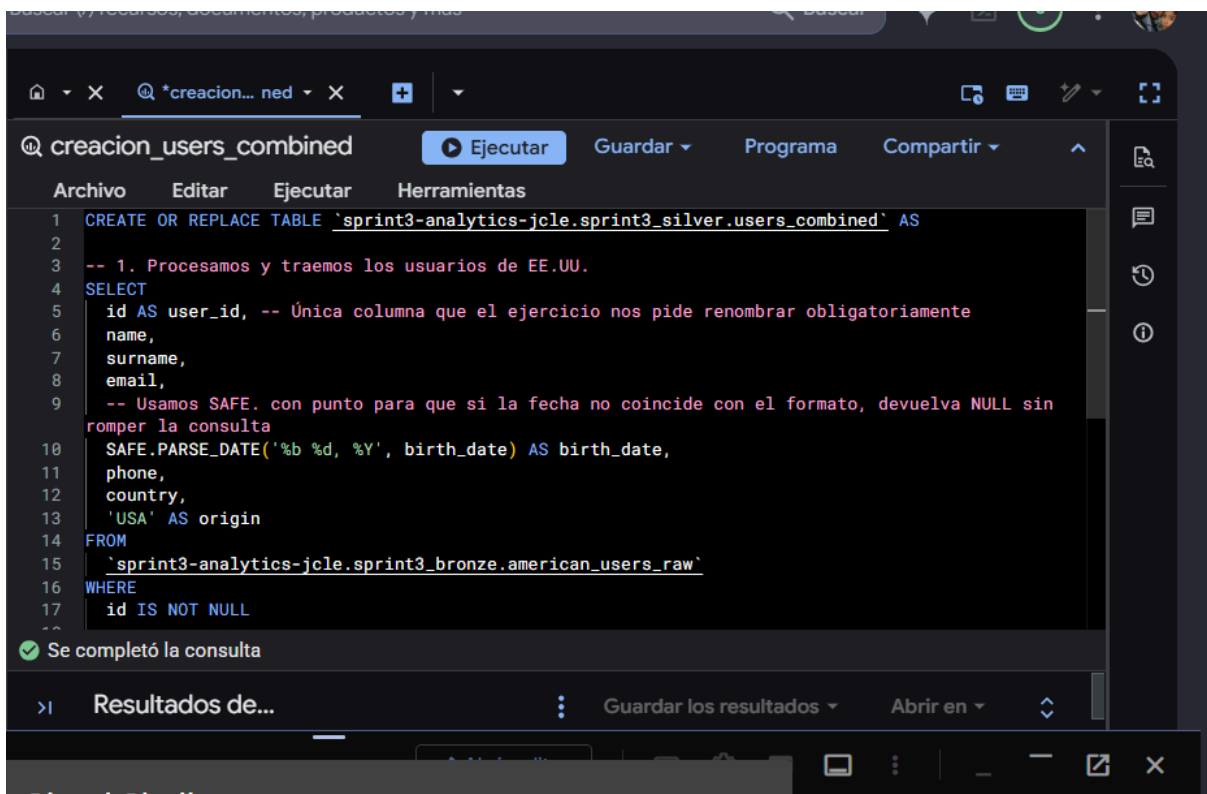
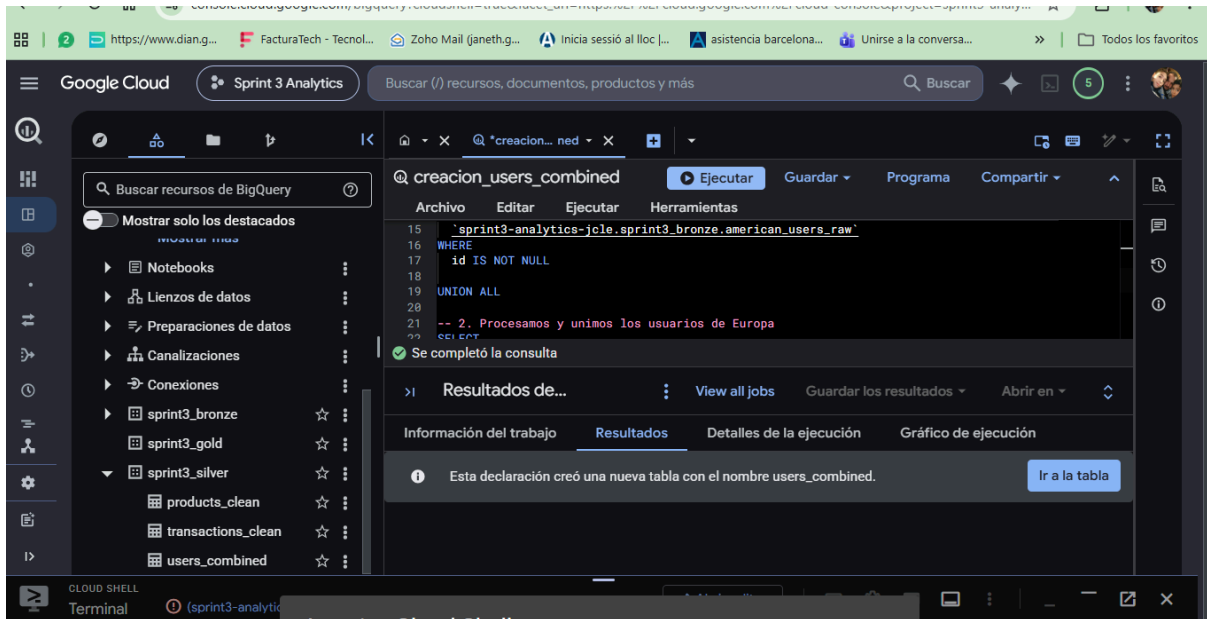
Ejercicio 3: Unificación de Usuarios (UNION)

Tenemos a los usuarios fragmentados por región.

Tarea:

Crea la mesa `sprint3_silver.users_combined`. Utiliza `UNION ALL` para unificar a los usuarios de EE.UU. y Europa en una única lista maestra. Añade una columna calculada `origin` para saber de dónde vienen.

» **Estandarización de Nombres** : La columna original id es ambigua; renombrarla a user_id.



Ejercicio 4: Materialización de Compañías y Tarjetas de Crédito

Escenario:

Para completar el modelo de datos en la capa Silver, nos faltan dos dimensiones clave que actualmente dependen de archivos CSV externos: **Compañías** y **Tarjetas de Crédito**. No podemos depender de archivos sueltos para el análisis final. Debemos importar estos datos en tablas nativas (Silver) y aprovechar para corregir formatos.

🧩 Tareas:

- » 1. Crea la tabla `sprint3_silver.companies_clean`. Copia los datos tal cual, pero asegúrate de que sea una tabla nativa de BigQuery (no una vista externa).
- » 2. Crea la tabla `sprint3_silver.credit_cards_clean`. Copia los datos tal cual, pero asegúrate de que sea una tabla nativa de BigQuery (no una vista externa).

Si es necesario, renombrar el id según corresponda.

Creación Tabla `companies_clean`:

Google Cloud Sprint 3 Analytics

Buscar (f) recursos, documentos, productos y más

Zona de pruebas Configura la facturación para disfrutar todas las funciones de BigQuery. Más información

Descartar Actualizar

Buscar recursos de BigQuery

Mostrar solo los destacados

sprint3-analytics-jcle

Consultas

Consultas (clásicas) (7)

Notebooks

Lienzos de datos

Preparaciones de datos

Canalizaciones

Conexiones

sprint3_bronze

sprint3_gold

creacion_com...

Ejecutar Guardar Programa Compartir

Archivo Editar Ejecutar Herramientas

```
1 -- 1. Creamos la tabla nativa de Compañías limpias
2 CREATE OR REPLACE TABLE `sprint3-analytics-jcle.sprint3_silver.companies_clean` AS
3 SELECT
4   id AS company_id,
5   company_name,
6   phone,
```

Esta secuencia de comandos procesará 0 B cuando se ejecute.

Tiempo transcurrido	Declaraciones procesadas	Estado del trabajo
4 s	2	✓ SUCCESS

Estado	Hora de finalización	SQL	Acción
✓	4:16 p.m. [2:1]	CREATE OR REPLACE TABLE `sprint3-analytics-jcl	Ver resultados
✓	4:16 p.m. [12:1]	CREATE OR REPLACE TABLE `sprint3-analytics-jcl	Ver resultados

CLOUD SHELL Terminal (sprint3-analytics-jcle)

Rechazar Autorizar

Información para identificar que es una tabla nativa:

creacion... ean x compani... ean x

sprint3-analytics-jcle / Datasets / sprint3_silver / Tables / companies_clean

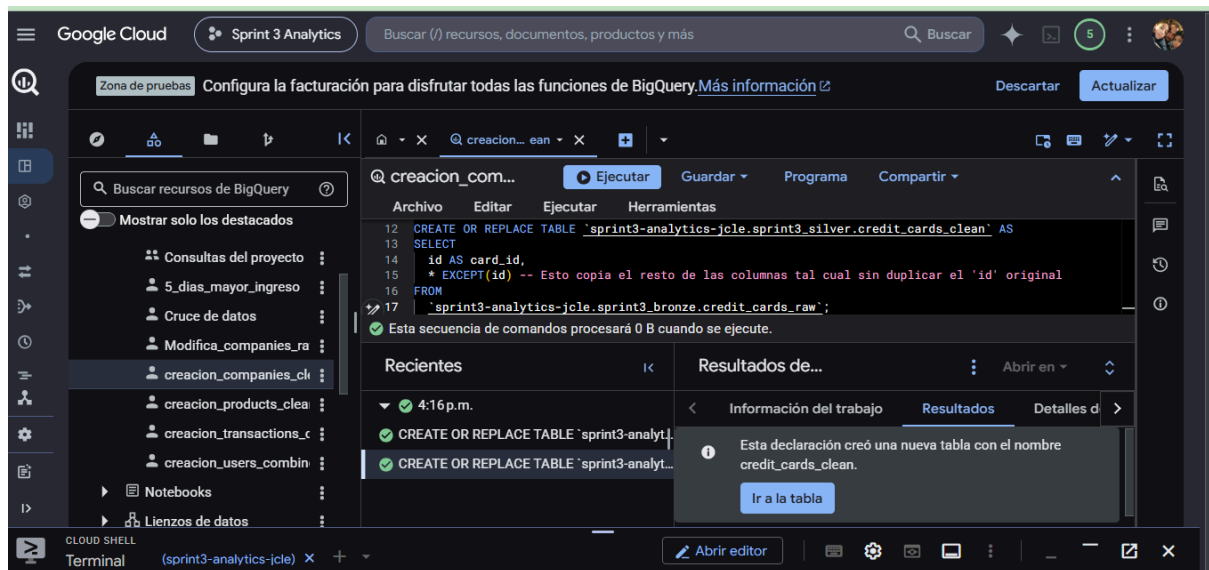
companies_cl... Consulta Chat Abrir en Compartir Copiar Instantánea Borrar Exportación

Esquema Detalles Vista previa Explorador de tablas Vista previa Estadísticas Linaje Perfil de datos Calidad de los datos

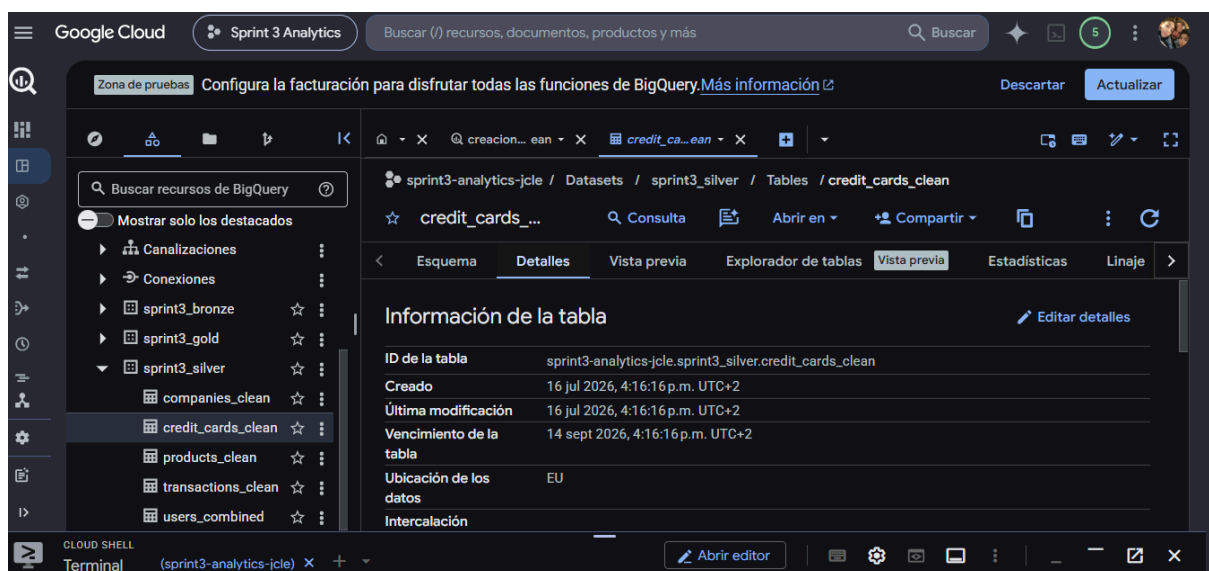
Información de la tabla Editar detalles

ID de la tabla	sprint3-analytics-jcle.sprint3_silver.companies_clean
Creado	16 jul 2026, 4:16:14 p.m. UTC+2
Última modificación	16 jul 2026, 4:16:14 p.m. UTC+2
Vencimiento de la tabla	14 sept 2026, 4:16:14 p.m. UTC+2
Ubicación de los datos	EU
Intercalación predeterminada	
Modo de redondeo predeterminado	ROUNDING_MODE_UNSPECIFIED
No distingue mayúsculas de minúsculas	false
Descripción	

Creación Tabla credit_cards_clean:



Información para identificar que es una tabla nativa:



Nivel 3: Presentación de Datos y Creación de Vistas

Escenario:

En el nivel anterior actuaste como Data Engineer limpiando los datos (_clean). Ahora toca ponerse el sombrero de Analista de Negocio. El problema actual es que las tablas están separadas y algunas tienen formatos difíciles de explotar (como la lista de productos comprimida en una sola celda). En este nivel crearás **vistas** para facilitar el acceso a la información y generarás informes finales.

🎯 Objetivos:

- » Crear Vistas (`CREATE VIEW`) para encapsular lógica de negocio compleja.
- » Resolver el reto de datos agrupados en strings (`SPLIT+ UNNEST`).
- » Exportar resultados para terceros (`Reverse ETL`).

Ejercicio 1: La Vista de Marketing (Lógica de Negocio)

Marketing necesita segmentar a los clientes corporativos, pero no saben hacer JOINS. Su tarea es dejarles la información preparada.

🧩 Tarea:

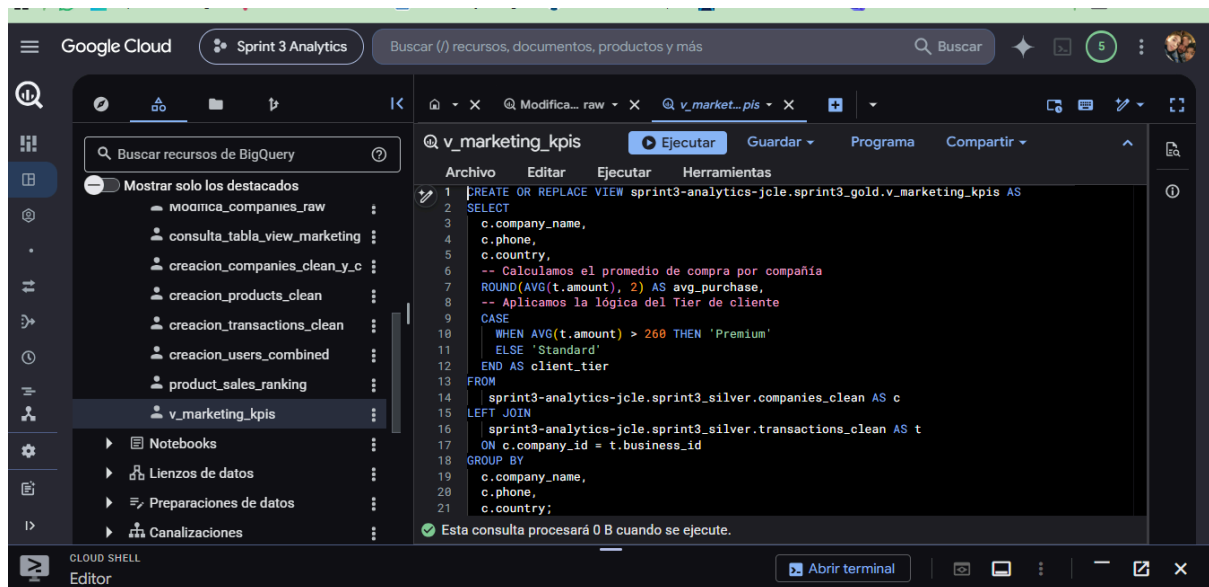
Creas una vista llamada `sprint3_gold.v_marketing_kpis` que muestre la siguiente información para cada compañía:

- » **Nombre de la compañía, Teléfono y País** (origen: `companies_clean`).
- » **Media de compra** (`AVG(amount)` de `transactions_clean`).
- » **Clasificación de Cliente (Lógica)** :
 - Crea una columna calculada llamada `client_tier`.
 - Si la media de compra es superior a **260 €**, etiqueta como **"Premium"**.
 - Si es igual o inferior, etiqueta como **"Standard"**.

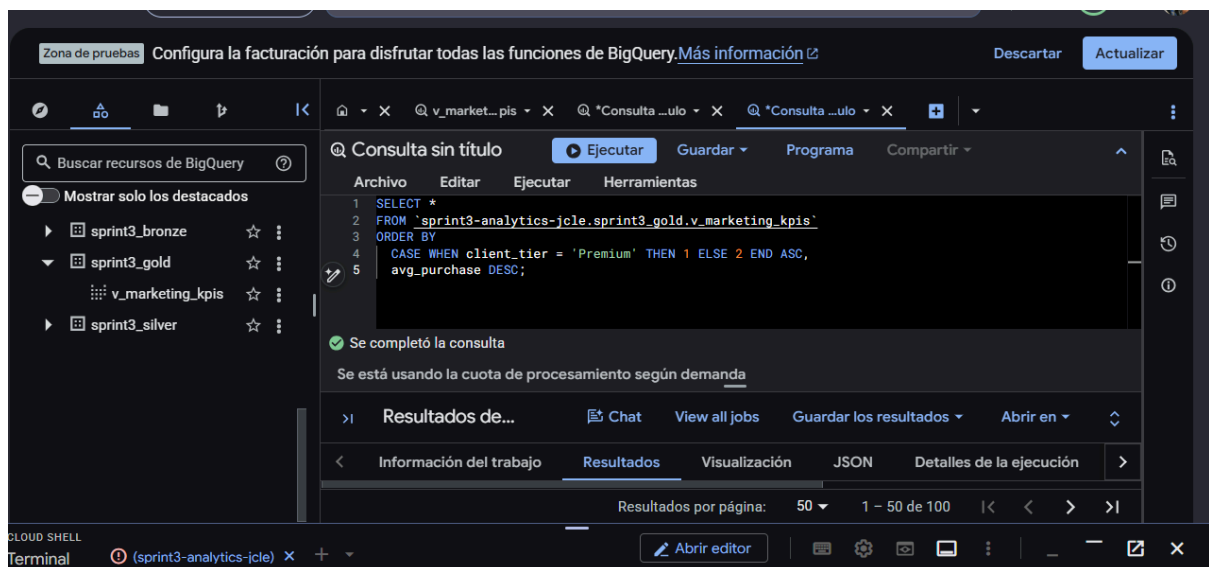
👉 Entrega:

Realiza una consulta `SELECT *` sobre tu nueva vista, ordenando los resultados para que aparezcan primero los clientes "Premium" y, dentro de éstos, los que tengan mayor promedio de compra.

Creación de la tabla View para Marketing:



Consulta SELECT *:



Resultados de la Consulta:

The screenshot shows the Google Cloud BigQuery interface. On the left, there's a sidebar with a search bar and a list of datasets: `sprint3_bronze`, `sprint3_gold`, `v_marketing_kpis`, and `sprint3_silver`. The main area displays the results of a query in a table format. The table has columns: `company_name`, `phone`, `country`, `avg_pur...`, and `client_tier`. There are 10 rows of data.

Fila	company_name	phone	country	avg_pur...	client_tier
1	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	284.87	Premium
2	Pretium Neque Corp.	07 77 48 55 28	eleifend.nec.malesuada@proto...	276.16	Premium
3	Urna Convallis Associates	06 01 24 77 04	aliquam@yahoo.org	274.24	Premium
4	At Associates	09 56 61 10 65	tristique.neque@yahoo.couk	272.21	Premium
5	Metus Vitae Associates	08 25 44 40 66	ad@protonmail.com	270.08	Premium
6	Aliquet Diam Limited	02 76 61 47 46	eu.eros.nam@icloud.org	269.6	Premium
7	Nec Luctus LLC	02 14 71 75 73	consectetuer.cursus@protonm...	268.6	Premium
8	Neque Tellus Incorporated	04 43 18 34 19	ultrices.iaculis.odio@aol.ca	267.85	Premium
9	Tortor Nunc Commodo Company	05 35 92 77 16	placerat.cras@hotmail.net	267.84	Premium
10	Cras Consulting	07 50 10 85 63	sed.consequat@google.ca	267.44	Premium

Ejercicio 2: Ranking de Productos (La Potencia de los Arrays)

Escenario de Negocio:

El equipo de ventas desea optimizar el catálogo. Necesitan un informe en la capa Gold que muestre el rendimiento real de cada producto. Gracias a tu buena ingeniería en la capa Silver, la mesa `transactions_clean` ya tiene los IDs de producto perfectamente organizados en una lista (Array). Ahora toca explotar esa estructura.

Objetivo:

Crea la tabla `sprint3_gold.product_sales_ranking` que contenga el inventario completo de productos y cuántas veces se ha vendido cada uno.

Requisitos del Informe:

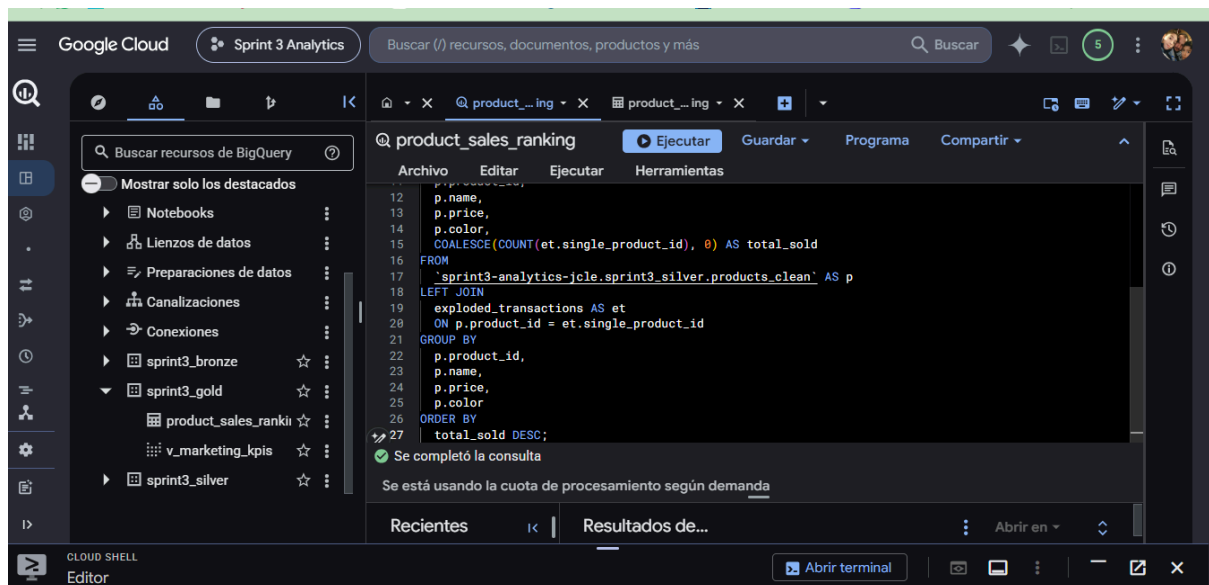
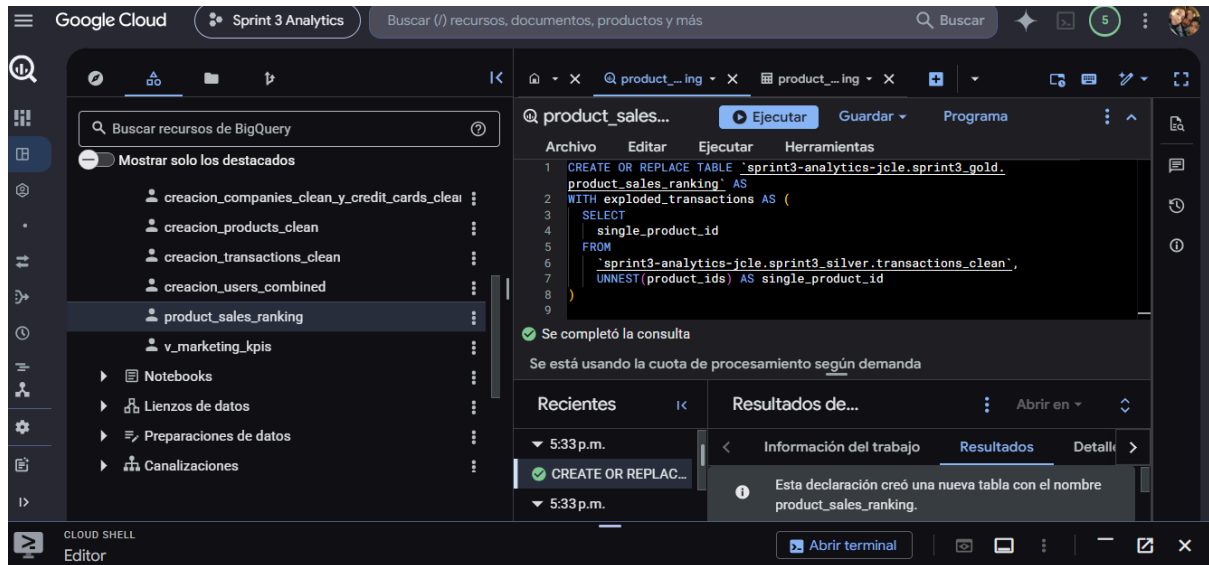
- » **Detalle del Producto** : Debe incluir `product_id`, `name`, `price` y `color` (vienen de la tabla `products_clean`).
- » **Métrica de Negocio** : Una columna nueva `total_sold` que cuente cuántas veces aparece este producto en las transacciones.
- » **Integridad** : Deben aparecer todos los productos, incluso los que tienen 0 ventas (quizás es necesario descatalogarlos).

Pista Técnica:

- » En `transactions_clean`, los productos están "encapsulados" dentro de un Array por cada transacción.

- » Primero necesitarás "allanar" la tabla de transacciones usando UNNEST(product_ids) para poder contar los productos individualmente.
- » Después, haz un cruce (LEFT JOIN) empezando por la mesa de productos para no perder aquellos que nunca se han vendido.

Estructura del script:



Resultado de la tabla que tendrá el equipo de ventas:

The screenshot shows the Google Cloud BigQuery interface. On the left, a sidebar lists various resources like Notebooks, Datasets, and Tables. The main area displays the 'product_sales_ranking' table with columns: product_id, name, price, color, and total_sold. The table contains 10 rows of data.

Fila	product_id	name	price	color	total_sold
1	52	riverlands the duel	31.52	#727272	2654
2	29	Tully maester Tally	167.2	#111111	2635
3	21	duel Direwolf	96.9	#e2e2e2	2609
4	16	the duel warden	180.91	#666666	2608
5	66	mustafar jinn	2.12	#545454	2601
6	87	sith Jade	96.26	#e2e2e2	2598
7	33	duel warden	127.09	#191919	2597
8	48	rock Renly in	21.53	#9e9e9e	2597
9	22	riverlands north	160.96	#545454	2609

Ejercicio 3: Exportación de Resultados

Tu manager no tiene acceso a BigQuery y quiere el listado de "Top Productos" en un Excel para una reunión.

Tarea:

Exporta los datos de la tabla `product_sales_ranking` a **Google Sheets** o descarga el archivo **CSV** localmente.

Entrega:

Una captura de pantalla donde se vea el archivo abierto (Excel/Sheets) con los datos correctamente formateados.

The screenshot shows a Google Sheets spreadsheet titled 'product_sales_ranking'. The data is imported from BigQuery and is displayed in a table format. The columns are: product_id, name, price, color, and total_sold. The table contains 10 rows of data.

product_id	name	price	color	total_sold
52	riverlands the du	31,52	#727272	2654
29	Tully maester Ta	167,2	#111111	2635
21	duel Direwolf	96,9	#e2e2e2	2609
16	the duel warden	180,91	#666666	2608
66	mustafar jinn	2,12	#545454	2601
87	sith Jade	96,26	#e2e2e2	2598
33	duel warden	127,09	#191919	2597
48	rock Renly in	21,53	#9e9e9e	2597
22	riverlands north	160,96	#545454	2609

Datos formateados:

1:1

fx product_id

	A	B	C	D	E	F	G	H	I
1	product_id	name	price	color	total_sold				
2	52	riverlands the duel	31,52	#727272	2.654,00 €				
3	29	Tully maester Tully	167,2	#111111	2.635,00 €				
4	21	duel Direwolf	96,9	#e2e2e2	2.609,00 €				
5	16	the duel warden	180,91	#666666	2.608,00 €				
6	66	mustafar jinn	2,12	#545454	2.601,00 €				
7	87	sith Jade	96,26	#e2e2e2	2.598,00 €				
8	33	duel warden	127,09	#191919	2.597,00 €				
9	48	rock Renly in	21,53	#9e9e9e	2.597,00 €				
10	23	riverlands north	169,96	#545454	2.593,00 €				
11	68	Stark Karstark	167,37	#333333	2.589,00 €				
12	88	Stannis warden south	167,15	#515151	2.587,00 €				
13	73	Dorne bastard	114,09	#848484	2.584,00 €				
14	28	chewbacca mustafar	127,44	#efefef	2.584,00 €				
15	4	warden south duel	71,89	#111111	2.584,00 €				
16	31	Lannister	85,02	#3f3f3f	2.580,00 €				
17	34	of north	128,48	#919191	2.580,00 €				
18	38	ewok sith	55,43	#666666	2.578,00 €				
19	54	dooku solo	13,14	#e5e5e5	2.575,00 €				
20	31	Winterfell Lannister	28,01	#848484	2.575,00 €				

product_sales_ranking

Columnas

Editar

product_id

name

price

color

total_sold

Filtros

Añadir

Ordenar

Añadir

Límite de filas

+ ≡ product_sales_ranking Extracto 1 Hoja 1

Recuento: 5